

```
import pandas as pd

import numpy as np

import re

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.svm import SVC

from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# Load Dataset (example: Amazon Reviews dataset)

data = pd.read_csv('customer_reviews.csv') # Replace with actual dataset


# Data Overview

print(f"Dataset Shape: {data.shape}")

print(data.head())


# Preprocessing

def preprocess_text(text):

    text = re.sub(r'[^\a-zA-Z]', ' ', text) # Remove non-alphabetic characters

    text = text.lower() # Convert to lowercase

    text = text.split() # Tokenization

    text = ' '.join(text) # Rejoin

    return text


data['cleaned_reviews'] = data['review_text'].apply(preprocess_text)
```

```
# Feature Extraction (TF-IDF)

vectorizer = TfidfVectorizer(max_features=5000)

X = vectorizer.fit_transform(data['cleaned_reviews']).toarray()

y = data['sentiment'] # Ensure sentiment column has Positive, Negative, Neutral


# Split Data

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Model Training & Evaluation

models = {

    'Logistic Regression': LogisticRegression(),

    'Support Vector Machine': SVC(kernel='linear'),

    'Naive Bayes': MultinomialNB()

}


for name, model in models.items():

    print(f"Training {name}...")

    model.fit(X_train, y_train)

    predictions = model.predict(X_test)

    print(f"{name} Accuracy: {accuracy_score(y_test, predictions):.2f}")

    print(f"Classification Report for {name}:

{classification_report(y_test, predictions)}")

    print("Confusion Matrix:")

    print(confusion_matrix(y_test, predictions))


# Example Prediction
```

```
sample_review = "The product quality is excellent!"

cleaned_sample = preprocess_text(sample_review)

sample_vectorized = vectorizer.transform([cleaned_sample]).toarray()

predicted_sentiment = models['Support Vector Machine'].predict(sample_vectorized)

print(f"Predicted Sentiment for Review: '{sample_review}' is {predicted_sentiment[0]}")
```